

Day	Work completed	Important decision / outcome	Reason
1	Reviewed final project requirements	Final Phronesis will combine a teacher-facing application with a substantial Data Science/ML component	Ensures the final system meets both the original Phronesis aims and the CM3005 Data Science template
1	Reviewed and prioritised functional requirements	Core analytics, student profiles, dashboard, explanations and recommendations remain central; large MIS integrations remain out of scope	Prevents feature creep and keeps development achievable
1	Designed final application data inputs	Phronesis will accept student, assessment, attendance, behaviour and homework data	Uses information schools can realistically obtain and minimises manual teacher input
1	Designed flexible assessment/attendance structure	Multiple assessments and attendance periods can be added instead of restricting the system to fixed columns	Allows longitudinal analysis and future updates
1	Considered sensitive/contextual information	SEN/FSM and similar characteristics will primarily be used for context/fairness rather than directly determining risk	Reduces potential bias and supports responsible educational AI
2	Selected ML training dataset	UCI Student Performance student-por.csv selected as the experimental training dataset	Contains 649 secondary-school records and known academic outcomes
2	Reviewed dataset variables	Identified academic/engagement variables and separated irrelevant or sensitive variables	Keeps the predictive model focused and interpretable
2	Defined prediction target	$G3 < 10$ = At Risk; $G3 \geq 10$ = Not At Risk	Provides a clear outcome for the classification experiment
2	Investigated target distribution	100 At Risk and 549 Not At Risk students identified	Revealed class imbalance that must be considered during evaluation
2	Performed initial exploratory comparison	Earlier grades, previous failures, absences and study time were examined between the two groups	Checks whether potential predictors show meaningful differences before modelling
2	Defined Experiment 1	Use failures, absences and study time without G1/G2	Tests whether academic risk can be detected earlier
2	Defined Experiment 2	Add G1 and G2 to the early-warning inputs	Tests how much previous assessment evidence improves prediction
2	Planned engineered features	Grade change, grade average, previous-failure indicator and low-study-time indicator	Allows testing whether derived information improves model performance
3	Set up Data Science project structure	Created separate folders for raw/processed data, notebooks, models, figures, results and development evidence while retaining the existing Phronesis <u>prototype</u>	Keeps experimental ML work organised without breaking the existing application and improves reproducibility
3	Loaded and inspected student-por.csv	Confirmed the dataset contains 649 student records and 33 variables	Verifies the downloaded dataset before any transformations or modelling
3	Performed data-quality checks	Confirmed 0 missing values and 0 duplicate rows	Ensures preprocessing decisions are based on the actual quality of the dataset rather than unnecessary cleaning

3	Validated numerical and categorical ranges	Checked age, study time, failures, absences and G1/G2/G3 ranges and confirmed key school/sex categories were valid	Identifies impossible or suspicious values before analysis
3	Created binary at_risk target	Converted final grade into G3 < 10 = At Risk; G3 ≥ 10 = Not At Risk	Converts the academic outcome into a clear binary classification problem
3	Analysed class distribution	Confirmed 100 At Risk (15.41%) and 549 Not At Risk (84.59%)	Identified substantial class imbalance and showed that accuracy alone will be insufficient for evaluation
3	Generated risk-distribution visualisation	Created a bar chart comparing At Risk and Not At Risk students	Provides visual evidence of the class imbalance for EDA/reporting
3	Analysed first-period grades	Median G1 was 12 for Not At Risk and 8 for At Risk ; produced a boxplot	Shows a strong difference in earlier attainment between the two outcome groups
3	Analysed second-period grades	Mean G2 was 12.31 vs 7.53 and median G2 12 vs 8 for Not At Risk and At Risk respectively; produced a boxplot	Shows that assessment performance later in the year provides a strong risk signal
3	Analysed previous academic failures	Created prevFailure; 9.29% of Not At Risk students versus 49% of At Risk students had a previous failure	Identified previous academic difficulty as a potentially strong early-warning feature
3	Analysed school absences	Mean absences were 3.49 vs 4.61 and medians 2 vs 2.5	Found that absences differ between groups, although much less strongly than grades or previous failures
4	Investigated relationship between G1 and G2	Found a strong positive correlation of r = 0.865 and produced a scatter plot	Shows that earlier assessment results are strongly related and should not be treated as independent evidence without consideration
4	Engineered assessment-change feature	Created grade_change = G2 - G1	Allows Phronesis to represent whether academic performance improved or declined
4	Evaluated grade-change feature	Mean change was +0.25 for Not At Risk and -0.25 for At Risk ; 27.69% vs 35% experienced a decline	Showed that grade change may provide some information but separates the groups less strongly than absolute G1/G2 performance
4	Created assessment-average feature	Created grade_average from G1 and G2	Provides a simplified representation of overall prior attainment for feature-engineering experiments
4	Created EDA summary table	Consolidated G1, G2, previous failure, absences and grade-decline findings	Produces a concise evidence record linking exploratory analysis to later modelling decisions
4	Refined operational feature selection	Removed studytime from the core Phronesis experiments because teachers would not normally supply reliable revision-hour data	Better aligns the experimental ML model with data the finished Phronesis system could realistically obtain with minimal teacher effort
4	Finalised Experiment 1	Failures + absences → At Risk prediction	Tests whether risk can be identified before assessment results are available
4	Finalised Experiment 2A	Failures + absences + G1 + G2 → At Risk prediction	Tests how much previous assessment information improves risk prediction
4	Finalised Experiment 2B	Previous-failure indicator + absences + grade average + grade change → At Risk prediction	Tests whether engineered, Phronesis-style summary features can represent assessment history effectively

4	Prepared ML experiment datasets	Experiment 1 = 649 × 3 , Experiment 2A = 649 × 5 , Experiment 2B = 649 × 5	Confirms all student records were retained while using different feature representations
4	Created reproducible train/test splits	Used 80% training / 20% testing , random_state=42 and stratification for all experiments	Ensures fair and reproducible model comparisons while preserving the class distribution
4	Verified split balance	Each experiment contains 519 training and 130 test students , with 15.41% At Risk in training and 15.38% in testing	Confirms stratification worked and each experiment will be evaluated on comparable data
5	Created majority-class baseline	Baseline predicted every student as Not At Risk and achieved 84.62% accuracy but 0% precision, recall and F1	Demonstrated that high accuracy can be misleading because the dataset is imbalanced
5	Evaluated baseline using multiple classification metrics	Added accuracy, precision, recall, F1 and confusion matrix rather than relying on accuracy alone	Provides a more meaningful evaluation of At Risk identification
5	Trained Logistic Regression on Experiment 1	Default Logistic Regression using failures and absences achieved 83.08% accuracy, 0% recall and 0% F1	Showed that these early-warning variables alone were insufficient for useful minority-class prediction with the default model
5	Trained Logistic Regression on Experiment 2A	Adding G1 and G2 increased performance to 92.31% accuracy, 72.73% precision, 80% recall and 76.19% F1	Demonstrated the substantial predictive value of previous assessment information
6	Trained Logistic Regression on Experiment 2B	Engineered representation achieved 93.08% accuracy, 73.91% precision, 85% recall and 79.07% F1	Produced the strongest Logistic Regression result on the current test split
6	Compared Logistic Regression experiments	Experiment 2B performed slightly better than 2A while Experiment 1 remained weak	Identified a promising feature representation but highlighted the need for further validation
6	Produced Logistic Regression confusion-matrix visualisations	Exp 1 detected 0/20 , Exp 2A detected 16/20 , and Exp 2B detected 17/20 At Risk test students	Provides visual evidence of how assessment information improves risk detection
6	Saved Logistic Regression evaluation evidence	Created comparison results and saved figures/results for later evaluation	Preserves reproducible evidence for later model comparison
7	Trained Decision Tree on Experiment 1	Achieved 84.62% accuracy, 50% precision, 25% recall and 33.33% F1	Decision Tree identified some At Risk students where Logistic Regression Exp 1 identified none
7	Trained Decision Trees on Experiments 2A and 2B	Both produced 89.23% accuracy, 65% precision, 65% recall and 65% F1	Assessment information again substantially improved prediction, while 2A and 2B produced identical test performance
7	Investigated Decision Tree feature importance	G2 dominated Experiment 2A while grade average dominated Experiment 2B	Provided initial insight into which academic variables the tree relied upon
7	Checked Decision Tree overfitting	Training accuracy was 98.65% for 2A and 98.46% for 2B , compared with 89.23% test accuracy	The gap suggested unrestricted Decision Trees were fitting the training data too closely
8	Trained Random Forest on Experiment 1	Achieved 83.85% accuracy, 45.45% precision, 25% recall and 32.26% F1	Confirmed that limited early-warning features remained weak even with a more complex ensemble model
8	Trained Random Forest on Experiment 2A	Achieved 89.23% accuracy, 65% precision, 65% recall and 65% F1	Assessment information again produced a substantial improvement

8	Trained Random Forest on Experiment 2B	Achieved 87.69% accuracy, 58.33% precision, 70% recall and 63.64% F1	Detected slightly more At Risk pupils than 2A but produced additional false positives
8	Compared three ML algorithms across all experiments	Logistic Regression Experiment 2B currently gave the strongest single-split result	Established a provisional best model while avoiding final model selection before stronger validation
8	Created master model comparison	Combined baseline, Logistic Regression, Decision Tree and Random Forest results into one results table	Allows direct comparison of algorithms, feature representations and classification metrics
8	Identified overall modelling pattern	Experiment 1 was weak across all algorithms, while assessment-informed Experiments 2A/2B performed substantially better	Suggests previous assessment performance is important for academic-risk prediction in this dataset
9	Applied 5-fold stratified cross-validation to Logistic Regression, Decision Tree and Random Forest using	Logistic Regression 2B produced the strongest overall average performance, while Random Forest 2B achieved the highest default recall	Provides a more reliable comparison than relying on one train/test split and tests model stability across multiple folds
9	Calculated mean and standard deviation for accuracy, precision, recall, F1 and ROC-AUC	Logistic Regression showed strong overall performance and stable ROC-AUC, while recall varied more between folds	Measures both average performance and consistency across validation folds
9	Compared raw assessment features with engineered features	Experiment 2B performed slightly better overall, but differences from 2A were small	Shows that engineered grade-average and grade-change features provide a useful alternative representation without overstating their benefit
9	Investigated class imbalance using balanced Logistic Regression and Random Forest models	Balancing substantially improved recall for At Risk students, especially for Random Forest	The dataset contains far fewer At Risk students, so class weighting was tested to reduce missed positive cases
10	Compared default and balanced model performance	Balanced Random Forest 2B achieved the strongest recall and improved F1, while default Logistic Regression retained higher precision	Demonstrates the trade-off between detecting more At Risk students and increasing false-positive alerts
10	Generated out-of-fold Logistic Regression probabilities for threshold analysis	Threshold evaluation was performed without using the final test set	Prevents test-set leakage and allows threshold decisions to be based on cross-validated training predictions
10	Evaluated classification thresholds of 0.30, 0.40, 0.50 and 0.60	A 0.40 threshold provided a strong balance between precision, recall and F1 compared with the default 0.50 threshold	Investigates whether Phronesis can detect more At Risk students without creating an excessive number of false alerts
10	Compared confusion matrices at different thresholds	Lowering the threshold from 0.50 to 0.40 identified four additional At Risk students while creating only two additional false positives	Provides a practical interpretation of the precision-recall trade-off in terms of actual student classifications
10	Investigated Decision Tree complexity using different maximum depths	A maximum depth of 2 produced substantially better recall, F1 and ROC-AUC than the unrestricted tree	Supports the earlier indication of overfitting and shows that controlling model complexity improves generalisation
10	Identified models to carry forward for final evaluation	Logistic Regression 2B, Balanced Random Forest 2B and depth-2 Decision Tree 2B remain the main candidates	Narrows the next stage to the strongest and most practically relevant models before final model selection and explainability work
11	Compared the final candidate models using pooled out-of-fold training predictions	Logistic Regression 2B at 0.40, Balanced Random Forest 2B and depth-2 Decision Tree 2B were compared using accuracy, precision, recall, F1 and ROC-AUC	Provides a fair comparison using predictions generated from cross-validation rather than relying on a single train/test split
11	Performed false-negative and false-positive error analysis on the strongest models	The models made different types of errors, with Logistic Regression producing fewer false positives while the Random Forest and Decision Tree detected more At Risk students	Helps evaluate the practical consequences of model errors rather than comparing metrics alone
11	Identified students missed by all three candidate models	Six At Risk students were missed by Logistic Regression, Random Forest and Decision Tree, with most having borderline previous attainment values	Shows that some errors are caused by limitations in the available predictive features rather than one particular algorithm

11	Added Average Precision as an additional evaluation metric	Logistic Regression produced the strongest Average Precision, while the tree and forest still provided stronger recall at their selected thresholds	Provides a metric focused on performance for the minority At Risk class and strengthens evaluation of the imbalanced dataset
11	Performed Decision Tree hyperparameter tuning using 5-fold cross-validation	Very shallow trees performed best and depth 2 was retained over depth 1 despite identical classifications because depth 2 produced stronger ROC-AUC and Average Precision	Reducing tree complexity helps control overfitting while retaining more useful ranking information
11	Tuned the Random Forest using GridSearchCV on training data only	The strongest configuration used 200 trees, no maximum depth, min_samples_leaf=10 and no class weighting	Hyperparameter tuning improved the balance between precision and recall without using the holdout test set for model selection
11	Generated out-of-fold predictions for the tuned Random Forest	The tuned forest achieved approximately 0.942 accuracy, 0.778 precision, 0.875 recall and 0.824 F1	Confirms that Random Forest tuning substantially improved performance compared with the earlier forest configurations
12	Performed Logistic Regression hyperparameter tuning	The strongest unscaled configuration used C=10 with balanced class weights, but its overall F1 did not outperform the existing 0.40 threshold Logistic Regression	Shows that hyperparameter optimisation does not necessarily produce a better operational model
12	Repeated Logistic Regression tuning using a StandardScaler Pipeline	Scaling changed the selected regularisation value to C=100 but the best mean cross-validation F1 remained approximately 0.7644	Checks whether differences in feature scales affected regularised Logistic Regression while preventing preprocessing leakage during cross-validation
12	Selected the three final model candidates	Logistic Regression 2B at 0.40, Decision Tree 2B depth 2 and tuned Random Forest 2B were retained for the final comparison	These represented the strongest combinations of precision, recall, F1, ranking performance and practical usability
12	Created and saved the final three-model comparison	Tuned Random Forest achieved the highest pooled OOF accuracy and F1, while retaining high recall and strong ROC-AUC and Average Precision	Provides evidence for selecting a final model using several performance measures rather than accuracy alone
12	Selected and froze the tuned Random Forest as the final model	The final configuration used 200 trees, no maximum depth, min_samples_leaf=10, no class weighting and a 0.50 classification threshold	Freezing the configuration before final evaluation prevents further tuning based on holdout performance
12	Evaluated the frozen Random Forest on the final holdout test set	The model achieved 88.46% accuracy, 58.06% precision, 90% recall, 70.59% F1, 0.932 ROC-AUC and 0.852 Average Precision	Provides a final check of how the selected model generalises to data not used for its final parameter selection
12	Analysed the final holdout confusion matrix	The model correctly identified 18 of 20 At Risk students, missing 2 while producing 13 false-positive alerts	Gives a practical interpretation of the model's 90% recall and shows the cost of prioritising detection of At Risk students
12	Calculated Random Forest built-in feature importance	grade_average was the dominant feature with approximately 82.7% of the total built-in importance	Identifies which predictors the final model relies on most heavily and improves model explainability
12	Performed permutation importance on the holdout data	Shuffling grade_average caused by far the largest reduction in F1, while the other features produced little change	Provides an additional explainability method and supports the finding that previous attainment is the model's dominant predictive signal
12	Evaluated probability quality using the Brier score	The final Random Forest achieved a Brier score of approximately 0.0639	Assesses the quality of the model's probability estimates rather than only evaluating its binary classifications
12	Performed a five-bin calibration analysis	Very low-risk predictions were well calibrated, but some middle probability ranges overestimated the observed proportion of At Risk students	Indicates that raw model probabilities should not be presented to teachers as precise probabilities of failure
12	Defined how model output should be represented within Phronesis	Risk output should be used primarily as a risk score or category rather than claiming an exact percentage probability of failure	Prevents the system from presenting model confidence with more certainty than the calibration evidence supports

Day

Work completed

Important decision / outcome

Reason

13	Designed the final Phronesis model adapter	Final model inputs were fixed as <code>prev_failure</code> , <code>absences</code> , <code>grade_average</code> and <code>grade_change</code>	Creates a clear bridge between normal school data and the four features expected by the trained Random Forest
13	Defined the final assessment data structure	Assessment records now include <code>student_id</code> , <code>subject</code> , <code>assessment_title</code> , <code>assessment_date</code> , <code>score</code> , <code>max_score</code> and <code>pass_mark</code>	Provides enough information to calculate model features without requiring teachers to manually transform their data
13	Defined assessment conversion onto the model's 0–20 scale	Assessment scores are converted using $(\text{score} / \text{max_score}) \times 20$	Keeps real school assessment data compatible with the 0–20 grade scale used during model training
13	Defined the rules for <code>grade_average</code> and <code>grade_change</code>	The latest two valid assessments for the same subject are used	Mirrors the two prior grade periods represented by G1 and G2 in the research dataset while avoiding comparisons between unrelated subjects
13	Defined the practical <code>prev_failure</code> proxy	<code>prev_failure</code> is set to 1 when an earlier assessment was below its pass mark	Provides a practical approximation of the UCI failures feature using information available within Phronesis
13	Defined the attendance-to-model conversion	<code>absences</code> is calculated as <code>total_sessions - sessions_attended</code>	Avoids incorrectly converting attendance percentages into absence counts without knowing the number of sessions
13	Defined missing-data behaviour	Students without the required assessment or attendance history receive Insufficient data for risk prediction	Prevents fabricated values such as zero being inserted into the model when evidence is missing
14	Implemented the assessment adapter	Assessment records are validated, converted to the 0–20 scale, grouped by student and subject, sorted by date and transformed into model features	Automates the feature engineering required before assessment data can enter the Random Forest
14	Added subject-specific assessment processing	Assessments are grouped by both <code>student_id</code> and <code>subject</code>	Prevents Phronesis from treating performance changes between different subjects as a genuine grade trend
14	Implemented calculation of <code>prev_failure</code> , <code>grade_average</code> and <code>grade_change</code>	Phronesis now generates the three assessment-derived ML features automatically	Removes the need for teachers to calculate machine-learning inputs manually
14	Added safe assessment eligibility handling	Students with fewer than two valid assessments are retained but their assessment features are marked as unavailable	Ensures incomplete students remain visible while preventing unreliable model predictions
14	Began separating model processing into adapter modules	Assessment, attendance and model-processing responsibilities were separated into dedicated files	Improves readability, maintainability and testing as the prototype develops into the final application
15	Refactored the assessment processing code	<code>process_assessments()</code> now uses the same validation logic as <code>validate_assessment_data()</code> instead of duplicating rules	Reduces duplicated code and means future validation changes only need to be made once
15	Completed assessment validation testing	30 assessment tests passed across valid, invalid, missing, boundary and non-numeric inputs	Provides evidence that assessment data is checked before feature engineering
15	Fixed future assessment date validation	A failed validation case showed that future assessment dates needed to be rejected, and the rule was corrected successfully	Prevents impossible future assessment results entering the prediction pipeline
15	Implemented the attendance adapter	Attendance records are validated and converted into <code>absences</code> and <code>attendance_percentage</code>	Produces the model's required absence-count feature while retaining a teacher-friendly attendance percentage
15	Completed attendance validation testing	All 15 attendance validation tests passed	Confirms invalid, missing and unrealistic attendance values are safely rejected

15	Designed and implemented the final feature merge	The student roster is retained as the master source while assessment and attendance features are attached using <code>student_id</code>	Prevents students disappearing simply because one data source is missing
15	Added prediction eligibility logic	Rows are marked Ready only when assessment and attendance data are valid and all four ML features are available	Ensures incomplete rows are never accidentally passed into the trained model
15	Tested model-feature merging	Complete data, incomplete assessment data, missing attendance, no data and multiple-subject cases were tested successfully	Confirms the adapter behaves safely across the main real-world integration scenarios
16	Connected the saved tuned Random Forest model	Phronesis loads <code>final_random_forest_2b.joblib</code> instead of retraining a model when the application starts	Ensures the final system uses the exact frozen model selected and evaluated during the ML stage
16	Implemented real ML risk prediction	Only Ready rows are sent to the Random Forest using the four features in the original training order	Maintains consistency between model training and final application prediction
16	Added safe handling for incomplete students	Students without enough information remain visible and receive Insufficient data instead of a prediction	Prevents misleading risk classifications based on incomplete records
16	Produced the first genuine Phronesis Random Forest predictions	S002 was classified At Risk , while the other complete sample students were classified Not At Risk	Confirms that the trained Random Forest is genuinely driving the final system's risk classifications
16	Added merge and prediction integration testing	Merge tests covered valid, incomplete, missing and multiple-subject cases, while prediction tests checked expected statuses and model outputs	Provides repeatable evidence that the complete model adapter and prediction pipeline behave correctly
16	Created <code>app_data.py</code>	A single <code>load_app_data()</code> function now performs data loading, feature processing, merging and model prediction	Prevents Streamlit pages from duplicating the same backend-processing logic
16	Connected the backend ML pipeline to the Streamlit application	The live app successfully displays Random Forest results from <code>load_app_data()</code>	Moves the project from standalone model testing into the actual final application
16	Replaced the prototype dashboard risk categories	The dashboard now uses At Risk , Not At Risk and Insufficient data instead of the old High/Medium/Low rule-based categories	Ensures the interface reflects the actual trained model rather than the original prototype scoring system
16	Added the new ML dashboard student table	The app displays student, subject, attendance, eligibility and Random Forest prediction while retaining incomplete students	Gives teachers a clear overview of actual model outputs and makes incomplete-data cases visible
17	Added downloadable CSV templates for core school data	Teachers can download the expected formats for subjects, classes, students, assessments and attendance before preparing uploads	Reduces formatting errors and makes Phronesis easier to use without requiring teachers to understand the internal data structure
17	Implemented Subject CSV upload	Teachers can bulk-add subjects using a CSV instead of entering every subject manually	Makes initial school setup faster and more practical for larger datasets
17	Added validation to Subject CSV uploads	Missing columns, blank subject names and duplicate/existing subjects are handled before new subjects are saved	Prevents malformed or duplicated subject records from entering the application data
17	Implemented Class CSV upload	Teachers can bulk-create classes containing class name, subject and year group information	Allows class structures to be created efficiently and links classes to existing subjects
17	Added dependency validation for uploaded classes	Uploaded classes are only accepted when their referenced subject already exists and year-group values are valid	Maintains consistency between related school records and prevents broken class-subject relationships

17	Implemented Student CSV upload	Teachers can upload multiple students and assign each student to one or more existing classes	Provides a scalable alternative to manually adding individual students
17	Added student-upload relationship checks	Student year groups are checked against the year groups of their selected classes and non-existent classes stop the upload	Prevents invalid student-class relationships and maintains consistent school structure
17	Implemented Assessment CSV upload	Teachers can bulk-upload assessment results containing student, class, assessment date, score, maximum score and pass mark	Allows real assessment histories to be added efficiently before the ML adapter generates model features
17	Added detailed assessment-upload validation	Required fields, missing data, numeric values and impossible score ranges are checked before assessment records are accepted	Protects the later feature-engineering and prediction pipeline from invalid assessment data
17	Implemented Attendance CSV upload	Teachers can bulk-upload total sessions and sessions attended for existing students	Allows Phronesis to generate the absences model feature from teacher-supplied attendance records
17	Added attendance-upload validation	Uploaded attendance checks required columns, numeric and whole-number session values, logical session ranges, valid student IDs and duplicate records	Prevents impossible attendance information from reaching the Random Forest pipeline
17	Integrated bulk upload alongside manual data entry	Teachers can now either enter school data manually through Phronesis or use downloadable templates and CSV uploads	Supports different teacher workflows while keeping both methods connected to the same underlying application data
17	Prepared CSV upload functionality for systematic testing	Core validation rules have been implemented, but thorough input and edge-case testing remains the next development task	Separates implementation from verification and ensures the final testing evidence reflects actual tested behaviour rather than assumed correctness

18	Began systematic Data Management testing after the bulk-upload implementation	Testing moved from feature creation to checking real user inputs, edge cases and failure handling	Provides evidence that the data-entry tools behave correctly rather than assuming that implemented validation works
18	Completed focused testing of Subject management	Manual add, rename, delete and CSV upload behaviour were checked, including empty values and duplicate subjects	Subjects are the first dependency for classes, so errors here would affect the rest of the school-data structure
18	Fixed Subject duplicate detection for inconsistent whitespace and capitalisation	Subject names are normalised before duplicate comparison	Prevents entries such as differently spaced versions of the same subject being stored as separate records
18	Confirmed dependency protection when deleting Subjects	A subject cannot be deleted while an existing class still references it	Prevents broken class-to-subject relationships
18	Continued using a test → fail → fix → retest workflow	Bugs found during testing are recorded before being corrected and retested	Produces clearer development evidence and shows how the system improved through testing
19	Completed focused testing of Class management	Class add, edit, delete and CSV upload flows were exercised using normal and invalid inputs	Checks that class setup remains reliable before students and results are attached
19	Fixed duplicate Class detection with extra internal spaces	Class names are cleaned before case-insensitive duplicate checks	Stops visually identical classes such as differently spaced versions of 7B Maths being created twice
19	Added protection against deleting a Class containing students	Deletion is blocked when class_students contains students assigned to the selected class	Prevents students being left linked to a class that no longer exists
19	Retained subject and year-group links when creating and editing Classes	Classes continue to store subject_id and year_group rather than duplicating subject information	Keeps the relational data structure consistent and avoids unnecessary duplication

19	Identified remaining class-integrity edge cases for later review	Testing highlighted that changes affecting class year group may need to be considered alongside already assigned students	Separates completed functionality from lower-priority integrity improvements instead of silently assuming every case is solved
20	Completed focused testing of Student management	Student add, edit and CSV upload workflows were tested with class assignments and validation rules	Students connect the class structure to assessment and attendance data, so relationship checks are essential
20	Enforced the rule that a student cannot be assigned to two classes for the same subject	Selected classes are checked by subject_id before the student is saved	Prevents duplicate subject enrolment from producing ambiguous subject-specific analytics
20	Improved Student and class-name whitespace handling	Extra spaces are removed from names and uploaded class lists before validation	Makes uploads more tolerant of realistic formatting mistakes without creating duplicate records
20	Kept duplicate student names valid while preserving unique student IDs	Students are identified by student_id rather than assuming names are unique	Reflects realistic school data where different students may share the same name
20	Ensured class_students is updated when Student class assignments change	Old class links are removed and the newly selected links are saved	Keeps student-to-class relationships synchronised after edits
21	Completed focused testing of Assessment management	Manual entry, editing, deletion and CSV upload were tested across valid and invalid assessment records	Assessment history directly produces three Random Forest features, so bad records must be rejected before prediction
21	Strengthened Assessment numeric and range validation	Scores, maximum scores and pass marks are checked for missing, non-numeric and impossible values	Prevents invalid marks from corrupting grade_average, grade_change and prev_failure
21	Strengthened Assessment date validation	Invalid dates and future assessment dates are rejected	Keeps longitudinal trends based only on plausible completed assessments
21	Improved duplicate Assessment detection	Duplicate checks use student, subject, cleaned assessment title and assessment date	Prevents the same result being entered twice while still allowing different assessments for the same student
21	Kept Assessment processing subject-specific	Assessment records continue to derive their subject through the selected class and subject relationship	Prevents results from unrelated subjects being combined into one academic trend
22	Completed focused testing of Attendance management	Attendance add, edit, delete and CSV upload flows were checked using valid and invalid records	Attendance supplies the absences feature used by the Random Forest and therefore requires reliable validation
22	Added detailed Attendance upload validation	Session values must be numeric whole numbers, total sessions must be positive and sessions attended cannot exceed the total	Prevents impossible attendance records from entering the analytics pipeline
22	Added duplicate and student-ID checks for Attendance uploads	Uploads reject repeated student rows, unknown student IDs and students who already have attendance records	Maintains one current attendance record per student and avoids conflicting inputs
22	Preserved teacher-friendly attendance information alongside model features	Phronesis calculates attendance percentage for display while the adapter calculates absences for the model	Separates the information teachers need to read from the representation required by the trained model
22	Brought the core Data Management area to a stable functional state	Subjects, classes, students, assessments and attendance can now be entered manually or uploaded with validation	Provides a complete input workflow for the final application before concentrating on analytics pages
23	Built the combined Class Overview workflow	Teachers select Year Group, Subject and Class before viewing class-level analytics	Creates a clear drill-down path and avoids showing unrelated students or subjects together

23	Added class summary metrics	The page reports class size, prediction readiness and At Risk information for the selected class	Gives teachers a quick overview before examining individual pupils
23	Added a selectable student table within each class	Teachers can select one student directly from the class table to open their detailed profile below	Removes the need for a separate disconnected Students page and keeps the analysis in context
23	Corrected class At Risk percentage calculation	At Risk percentage uses students with prediction_status Ready as the denominator	Prevents students with Insufficient data from distorting the model-risk percentage
23	Converted internal model grades to teacher-friendly percentages in the interface	0–20 grade_average and grade_change values are displayed as percentage equivalents	Keeps the ML representation internal while presenting familiar school measures to teachers
24	Expanded the individual student profile inside the Class page	Selected students now show prediction status, risk output and key academic and engagement indicators	Brings the identify → understand → recommend workflow into one screen
24	Implemented rule-based student Insights	Attendance, academic performance, grade change, previous failure, homework and behaviour can generate contextual observations when data is available	Helps teachers interpret the wider student picture without claiming that every contextual factor drove the Random Forest
24	Implemented student Recommendations	Recommendations map identified concerns to practical follow-up actions such as attendance review, academic support and monitoring	Turns analytics into teacher-facing decision support rather than only displaying a prediction
24	Added editable parent communication	Phronesis can generate a draft parent email from the selected student's current information for the teacher to review and edit	Supports follow-up while keeping the teacher responsible for the final communication
24	Clarified the boundary between model inputs and contextual indicators	The Random Forest remains limited to prev_failure, absences, grade_average and grade_change; homework and behaviour are supporting context only	Maintains methodological consistency with the trained model and avoids misleading explanations
25	Completed the main Dashboard structure	The Dashboard now provides an overall summary followed by priority students, insights, class summaries and charts	Creates a low-cognitive-load starting point for teachers after signing in
25	Added Students Requiring Attention	At Risk predictions are surfaced in a priority table with class, subject, attendance and assessment context	Makes the most actionable cases visible without requiring teachers to open every class first
25	Added Quick Insights for attendance, academic, homework and behaviour concerns	Contextual concern counts are shown only when the relevant data is available	Provides a wider overview while handling optional data safely
25	Added My Classes summary and filters	Class-level risk, insufficient-data, attendance and assessment information can be filtered by year group and subject	Allows teachers to compare their classes and move from whole-workspace overview to a specific class
25	Added class risk and attendance charts	The Dashboard visualises classes with the highest At Risk proportions and the lowest average attendance	Provides a compact visual comparison while retaining the text-first dashboard design
26	Implemented teacher account creation and sign-in	A teachers.csv account file stores teacher ID, name, email, password hash and tutorial status	Adds authentication so Phronesis can provide a protected teacher workspace
26	Added separate Sign In and Create Account flows	Users must authenticate before the normal Dashboard, My Classes, Data Management and Settings navigation is available	Prevents application pages and class data from appearing on the public login screen
26	Added a first-login tutorial	New teachers see a short introduction to Dashboard, Classes, Data Management and recommendations before entering the app	Improves onboarding and explains the intended workflow before users begin uploading data

26	Added Settings and account controls	Teachers can update their name, change password, reopen the tutorial and log out	Provides the basic profile and session controls expected in the final application
26	Improved failed-login feedback	Unrecognised credentials return a clear email/password-not-recognised message and ask the user to try again	Gives useful feedback without exposing whether a specific email address exists
27	Identified shared-data leakage between different teacher accounts	Authentication originally changed who was logged in, but every account still read the same global data/new CSV files	Showed that login alone was not enough to create independent teacher workspaces
27	Designed a teacher-specific data-folder structure	Each teacher now uses a folder such as data/users/T001 containing their own students, subjects, classes, relationships, assessments and attendance files	Separates teacher-owned application data while retaining the existing simple CSV-based prototype architecture
27	Created user_data.py helpers	get_user_file() resolves paths from the logged-in teacher_id and create_user_files() prepares empty CSV files with the correct headers	Centralises workspace path handling and avoids repeating account-folder logic across the application
27	Changed app_data.py to load the logged-in teacher's files	Application analytics now use get_user_file() instead of fixed data/new paths for teacher-owned data	Ensures the Dashboard and Class analytics operate on the current teacher's workspace
27	Converted Data Management modules to teacher-specific reads and saves	Subjects, Classes, Students, Assessments and Attendance were updated so both manual edits and CSV uploads use get_user_file()	Prevents one teacher's changes from intentionally writing into another teacher's application data
28	Tested the application with a newly created empty teacher workspace	The new account exposed an empty-data edge case in the analytics pipeline	Testing with a realistic first-use state revealed behaviour that the original populated test dataset could not expose

28	Diagnosed a KeyError on student_id during model-feature merging	An empty assessment dataset produced an empty assessment_features DataFrame with no column names	The merge expected student_id even when there were zero assessment rows
28	Fixed empty Assessment feature output	process_assessments() now constructs its result with the expected student_id, subject, prev_failure, grade_average, grade_change and assessment_status columns even when there are no rows	Allows the model pipeline to handle a brand-new account without crashing during the merge
28	Confirmed the application runs after the empty-data fix	The teacher-specific account setup can now reach the application without the previous student_id KeyError	Provides a stable baseline for continuing final testing and evidence collection
28	Recorded authentication and workspace separation as the current development baseline	Future work can build on the completed login, tutorial, settings and user-specific data architecture rather than returning to the shared-data prototype	Reduces the risk of later changes accidentally undoing completed account and data-isolation work
29	Improved the Assessment CSV template workflow	Assessment templates are now generated for a teacher-selected class rather than being downloaded as a completely blank file	Reduces manual setup and prevents teachers needing to remember internal student IDs
29	Added student names alongside student IDs in Assessment templates	Each template row contains both student_id and the corresponding student name	Makes generated IDs understandable to teachers while continuing to use the unique ID internally
29	Added an Assessment template setup popup	Teachers choose the class and enter assessment title, date, maximum score and pass mark before downloading the template	Allows information shared by the whole assessment to be entered once instead of repeatedly for every student
29	Prefilled Assessment template information	Student ID, student name, class, assessment title, assessment date, maximum score and pass mark are automatically populated	Minimises repetitive data entry and reduces the chance of inconsistent assessment information
29	Simplified the teacher's Assessment CSV task	Score is deliberately placed as the final blank column so the teacher only needs to enter each student's result before uploading	Creates a quicker and more realistic workflow for entering whole-class assessment results
30	Improved the Attendance CSV template workflow	Attendance templates can now be generated for a selected class through a popup	Makes attendance entry consistent with the improved Assessment workflow
30	Added student and class information to Attendance templates	Student ID, student name and class name are automatically populated for every pupil in the selected class	Removes the need for teachers to look up student IDs or manually recreate their class roster
30	Added Total Sessions to the Attendance template setup	The teacher enters the total number of sessions once before downloading and the value is copied to every student row	Avoids repeatedly entering the same attendance-period information

30	Simplified the teacher's Attendance CSV task Added save handling for classes with no students when generating templates	sessions_attended is the final blank column, leaving only each student's attended-session value for the teacher to complete Template generation stops and informs the teacher when the selected class contains no students Assessment and Attendance downloads now prepare most of the required information automatically before the teacher edits the CSV	Reduces teacher workload while retaining the information required to calculate absences and attendance percentage Prevents empty or misleading CSV templates being downloaded Improves usability and better reflects the aim of minimising unnecessary teacher data entry
----	--	--	---

Day	Work completed	Important decision / outcome	Reason
31	Redesigned the public Phronesis landing page	Added a large branded hero area, classroom feature section, Why Phronesis cards, review	Gives the finished system a more professional first impression before teachers sign in
31	Improved public-page interaction and visual	Added rounded coloured cards, hover animations, shadows, wider layouts and	Makes the interface feel more polished and easier to visually scan
31	Removed fragile custom HTML from the landing	Public content now uses native Streamlit components styled through keyed containers	Prevents HTML being displayed as text and keeps the interface more reliable
31	Added clearer public explanation of Phronesis	Landing page explains how assessment, attendance and behaviour information	Helps users understand the purpose of the system before creating an account
31	Separated page-specific styling	About, login, tutorial and logged-in application styling are being separated into dedicated CSS	Reduces CSS conflicts and makes the interface easier to maintain
31	Began redesigning the Sign In/Create Account	Authentication is being updated to match the same Fredoka/Nunito, rounded and warm-	Creates a consistent transition from the public site into the Phronesis application

Day	Work completed	Important decision / outcome	Reason
32	Redesigned the Data Management interface	Subjects, Classes, Students, Assessments and Attendance were updated to use the same	Creates a consistent interface and makes data-entry tasks easier to understand
33	Standardised Data Management actions and	Add, edit, delete, template download and CSV upload actions were reorganised into	Reduces clutter and makes repeated workflows more predictable for the teacher
34	Redesigned the Class Overview page	Added class filters, summary metrics, student profile sections, risk status, indicators,	Makes the identify-understand-support workflow clearer and easier to follow
35	Improved analytical table presentation	Added risk-based row highlighting for student analysis tables and kept Data Management	Helps important risk information stand out without making administrative tables visually
36	Redesigned the Dashboard	Added compact summary cards, quick insights, priority student information, class	Makes the main dashboard more useful as a quick overview and reduces unnecessary
37	Improved dashboard visualisations and colour	Replaced repetitive default charts with themed Altair visualisations, hover	Makes trends and risk information easier to interpret while keeping the interface
38	Improved accessibility and completed final	Added the OpenDyslexic font option, refined page-specific CSS, fixed final layout issues and	Improves accessibility and brings the prototype to a stable state ready for